

Tutorial: Multi-Dataflow Composer Tool

CGR Coprocessor Generation port on Kria KV260 from Dataflow Networks

This tutorial shows how to generate a processor-coprocessor AXI-based system for Xilinx FPGA using the MDC tool (<http://sites.unica.it/rpct/>), and how to port it on Kria KV260 (<https://github.com/mcheshfar/AXI-based-driver-applications>).

The multi-dataflow network performs two functionalities for edge detection: Sobel and Roberts operators. Both algorithms consider the convolution of two kernels with an grayscale image (as shown in Fig. 1), in order to highlight the high-frequency variations due to the horizontal and vertical edges.

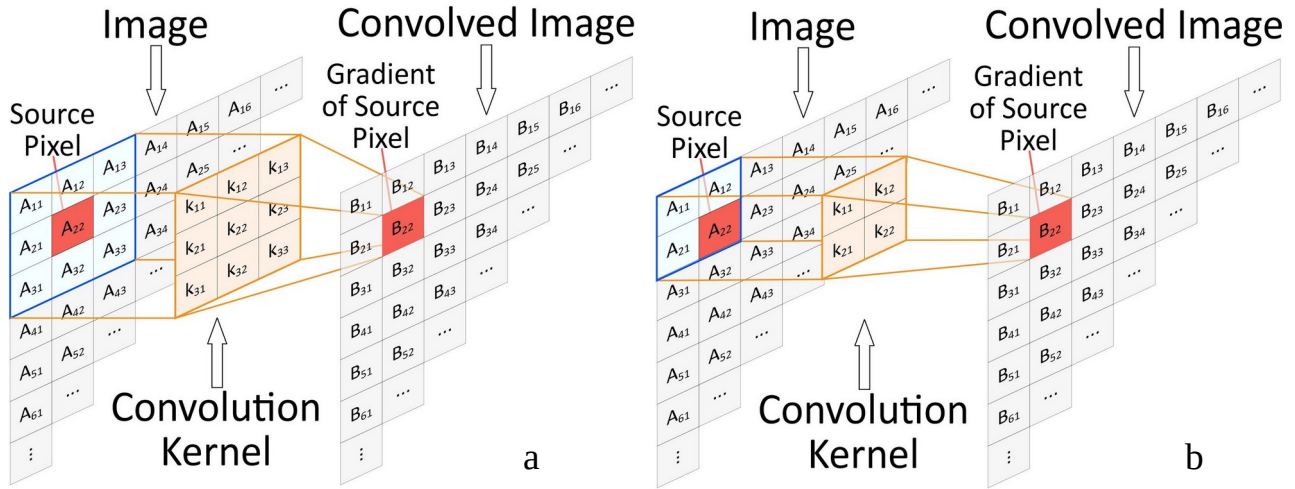
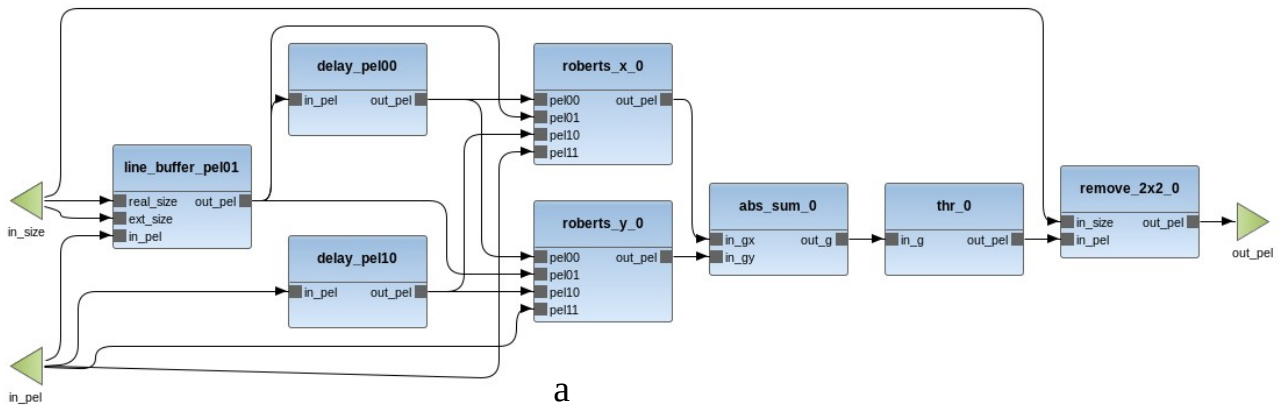


Figure 1: Computation of the convolution of kernels (x and y) of the Sobel (a) and Roberts (b) operators with an input image (A).

Figure 2 illustrates the single networks for Sobel (a) and Roberts (b) algorithms.



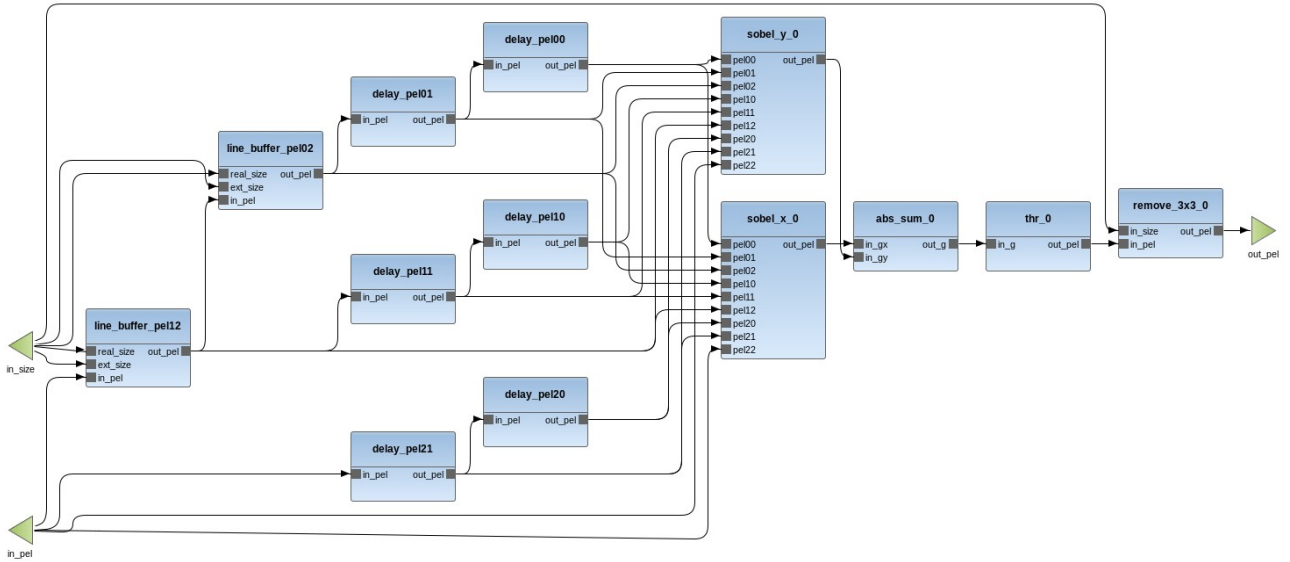


Figure 2: Dataflow description of the single networks for the Sobel (a) and Roberts (b) operators and for the multi-dataflow reconfigurable system (c).

The single networks and HDL component libraries have been created using CAPH tool(<http://caph.univ-bpclermont.fr>). CAPH is a framework for the specification, simulation and implementation of stream processing applications based on a dynamic Dataflow MoC ([ref](#)).

Necessary tools:

- MDC tool (to install this tool without a provided executable file, please, follow installation [guide](#));
- [Java Runtime Environment](#) (v8 or higher);
- [AMD Vivado tool](#). (Tutorial has been tested with Vivado 2024.2);
- A Xilinx FPGA board, Kria KV260.
- [MDC Example files](#)

b

1. Merging Process

Run MDC tool. (in Windows launch MDC executable, in Linux run it by eclipse)

Create a new run configuration: Run > Run configurations... , right click on Orcc compilation then select New. Now, chose a name for the configuration, and select Tutorial_EdgeDetection project, MDC backend and the output folder path, Figure 3.

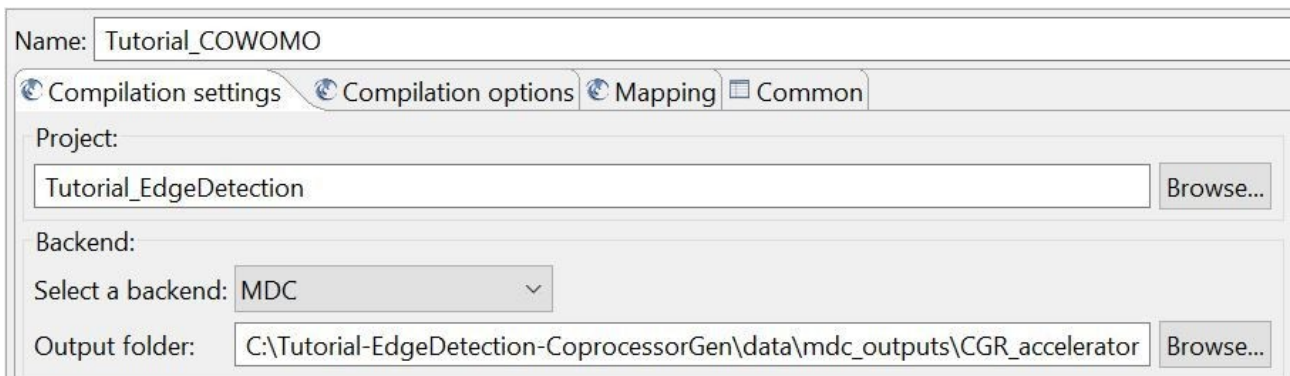


Figure 3 run configuration

Then select the two input dataflow networks: edgeDetection.roberts and edgeDetection.sobel and choose the merging algorithm (EMPIRIC or MOREANO, [ref](#)). In this tutorial we adopt the EMPIRIC one ,Figure 4.

Options:

☒ List of Networks to be Compiled and Merged

Number of Networks:

XDF List of Files:

Merging Algorithm

▾

☒ Generate RVC-CAL multi-dataflow

Figure 4: merging options

Tick “Generate RVC-CAL multi-dataflow”. And Run.

This step merges the two input dataflow networks, through the selected datapath merging algorithm. Click on Run.

Refresh the project folder to visualize the output folder with the generated multi-dataflow. In the generated networks you can notice as actors are shared, and functionalities of the two input networks are guaranteed by the insertion of the switching boxes as shown in Figure 5.

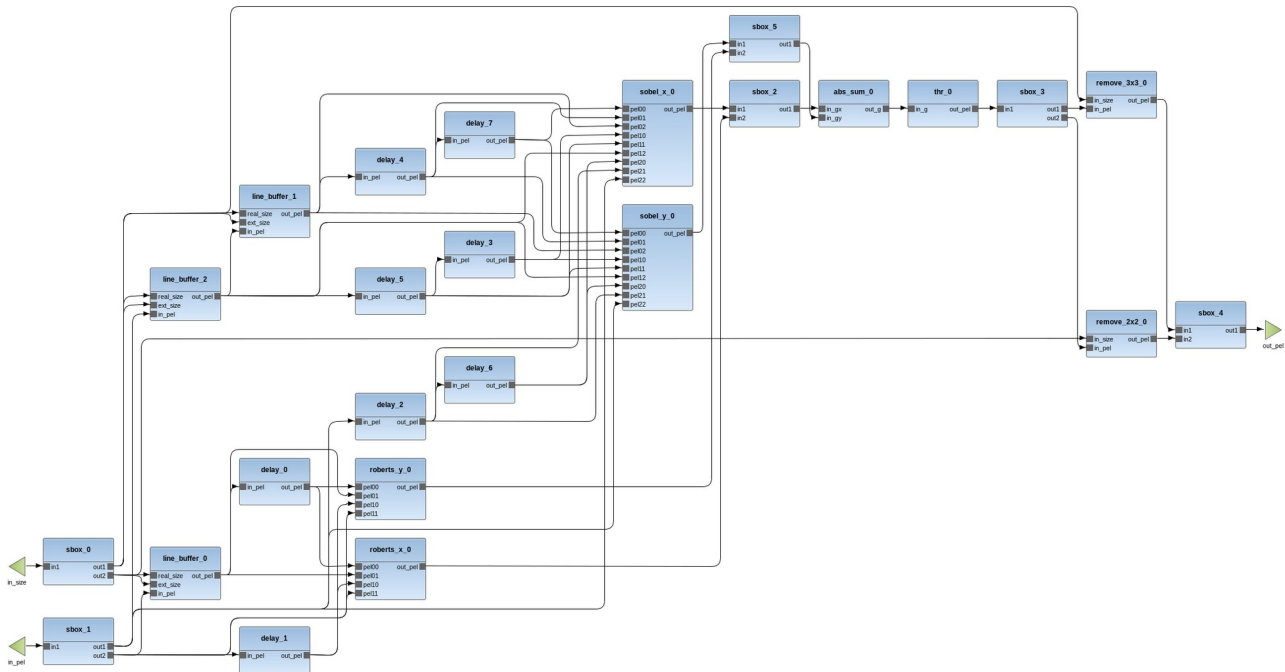


Figure 5: merged network

2. HDL Generation

Open again the configuration window: Run > Run configurations... Choose previous configuration, unselect “Generate RVC-CAL multi-dataflow” and tick “Generate HDL multi-dataflow” as Figure 6.

Then choose the protocol file and the HDL component Library.

1. Protocol file is in Tutorial-EdgeDetection-CoprocessorGen/data/mdc_inputs/protocol/ ([link](#)) folder.
2. HDL component library is in Tutorial-EdgeDetection-CoprocessorGen/data/mdc_inputs/HDL_compLib ([link](#)) folder.

The HDL component library has to contain all of the necessary hdl files. If some file, related to a specific library are needed, the should be put in: Component_Library_Folder/lib/libName folder.

Please pay attention that libName folder name has to match the library name!

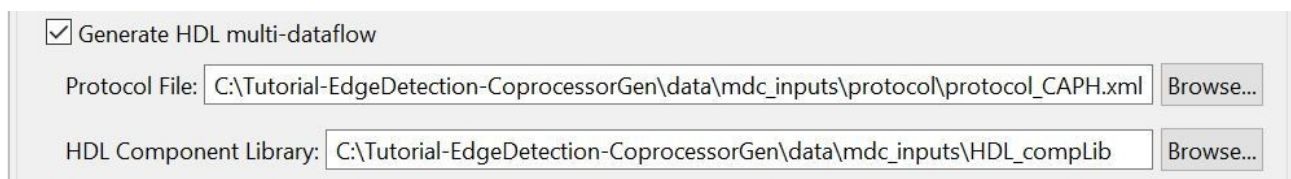


Figure 6: The HDL component library options

Output folder

Output folder contains one folder and two files:

1. HDL: includes all of the necessary files to create simulate and synthesize our CGR accelerator.

It contains also two files:

1. configNetID.txt - reports the ID value associated to each input dataflow.
2. report.txt - reports:
 - a) the number of actors of each input network
 - b) number of merged networks,
 - c) number of actors
 - d) number of original actors
 - e) number sbox actors
 - f) number of shared actors

3. Coprocessor Generation

To generate the Ip, you need to choose, Figure 7:

1. The processor-coprocessor coupling ([ref](#)), memory-mapped or stream . In this tutorial the **stream** is chosen.
2. The Host Processor. In this tutorial we use the **ARM** Processor.
3. If you want to use the DMA module ([ref](#)). We **tick** this box.
4. Kind of Xilinx board. In this tutorial the **Kria KV260** Vision AI Starter Kit
5. Kit is adopted:
 - a) Board Part: xilinx.com:kv260_som:part0:1.4
 - b) Partname: xck26-sfvc784-2LV-c

☐ Power Optimization
☒ System Generation
☒ Generate Accelerator IP
 Processor-Coprocessor Coupling
 STREAM
 Host Processor
 ARM
☒ Adopt DMA
 Board Part: xilinx.com:kv260_som:part0:1.4
 Partname: xck26-sfvc784-2LV-c
☐ ARTiCo³ Backend
☐ Enable Monitoring
☐ Pulp Backend
☐ Enable Profiling

Figure 7: Coprocessor Generation options

Output folder

Output folder contains two folders:

1. s_accelerator:
 - a) hdl: includes all of the necessary files to create the Custom IP in Vivado.
 - b) drivers: include the .c and .h files necessary to easily communicate with accelerator from the host processor (**we don't use them in this version.**).
- 2) Scripts: contains two scripts:
 - a) generate_ip.tcl - uses inputs in s_accelerator to create a project and package a Custom IP.
 - b) generate_top.tcl - create a top project, where the IP is instantiated and connected to Host Processor, using the necessary logic according to user requirements. (**we don't use them in this version.**)

By default these scripts consider as root the folder where s_accelerator and scripts folder are saved.

(you can find compatible version with this tutorial [here](#)) If the older version of MDC used, it needs some manually updates in s_accelerator.v in hdl folder:

In line 169: `assign in_pel_push = s00_axis_tvalid;` → `assign in_pel_push = s00_axis_tvalid & s00_axis_tready;`

In line 175: `assign in_size_push = s01_axis_tvalid;` → `assign in_size_push = s01_axis_tvalid & s01_axis_tready;`

In line 179: `assign m00_axis_tstrb = 4'b1111;` → `assign m00_axis_tstrb = 4'b1111;`

Change any **tstrb** to **tkeep**. (in lines 35, 44, 53, 167)

If vivado is launched in that folder, they don't need any modification. If Vivado is launched in a different folder (e.g Windows users), the users should open the scripts and replace root path “.” (set root “.”) with were necessary folders are saved.

4. Processor - Coprocessor System Generation

Launch Vivado.

Change directory to the root folder of the generated scripts,
as: `.../data/mdc_outputs/CGR_coprocessor`

Now, in the main menu (only in Linux environment): Tools → Run tcl Script... and choose
`.../CGR_coprocessor/scripts/generate_ip.tcl`. (Figure 8)

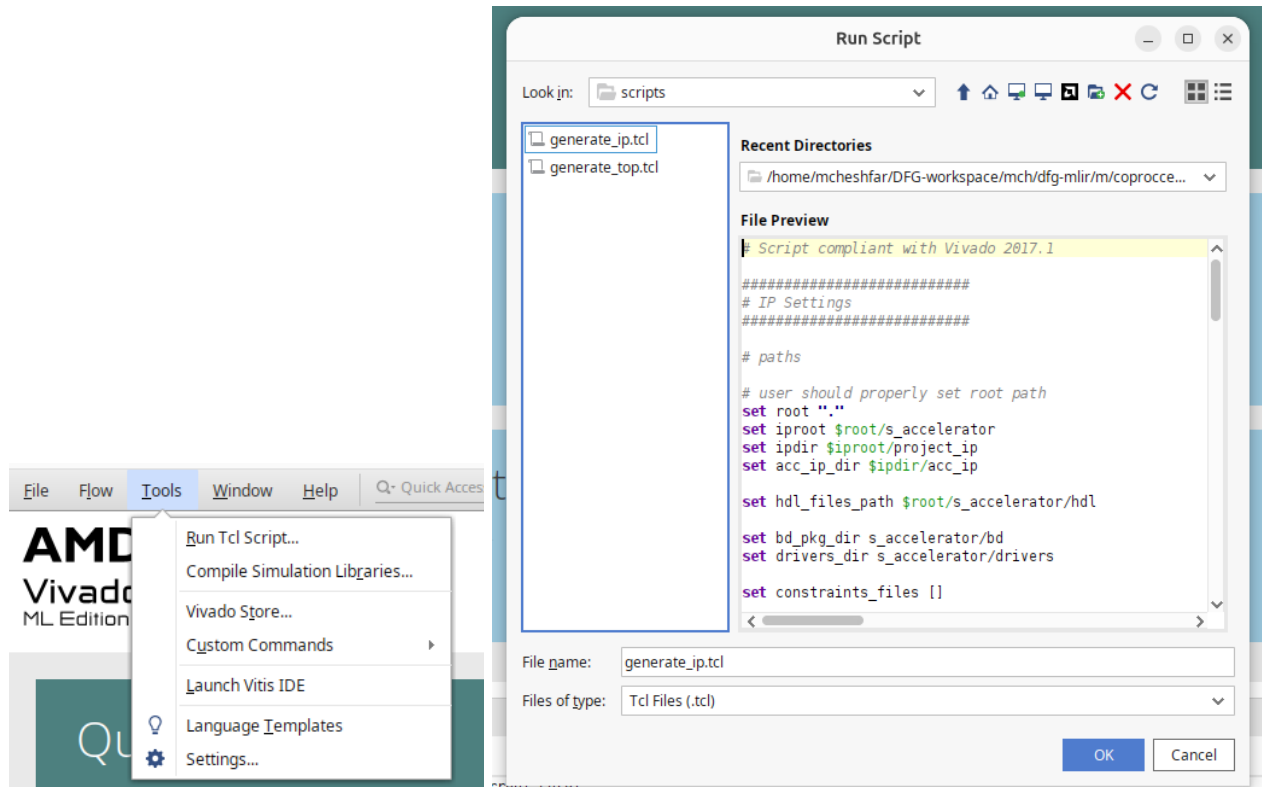


Figure 8: select tcl to run

When the IP is generate, it is possible to see the HDL files in the Source panel. You can see the IP pack in `s_accelerator/project_ip/acc_ip`. Copy `acc_ip`.

For coprocessor system design TCL file from the [ref](#) (`design_1.tcl`). Can you make a folder and put this TCL beside the `acc_ip`.

Inside this folder Open Vivado in TCL mode: `vivado -mode tcl`

In the TCL Console, run: `source design_1.tcl`

It make bitstream file automatically. You can file **design_1_wrapper.bit** in `myproj/project_1.runs/impl_1`.

Open the Vivado project: `vivado ./myproj/project_1.xpr` (Figure 9)

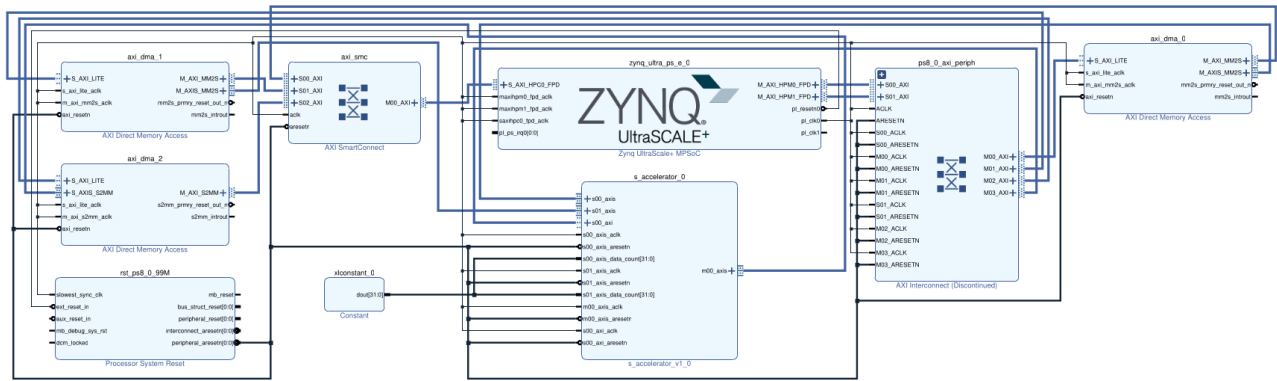


Figure 9: Block design for coprocessor

5. OS-ORIENTED ACCELERATION MANAGEMENT ON KRIA KV260

1. [on your laptop] SD Flashing ---> [KV260 Ubuntu 22.04 image](#)



Figure 10: flash SD card with application on PC

2. Connect the board (via Ethernet or USB, or with keyboard/screen), insert the SD, and turn the board on

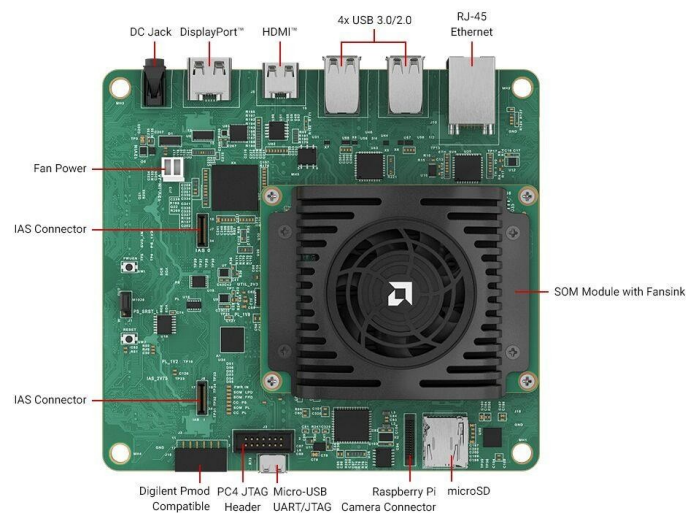
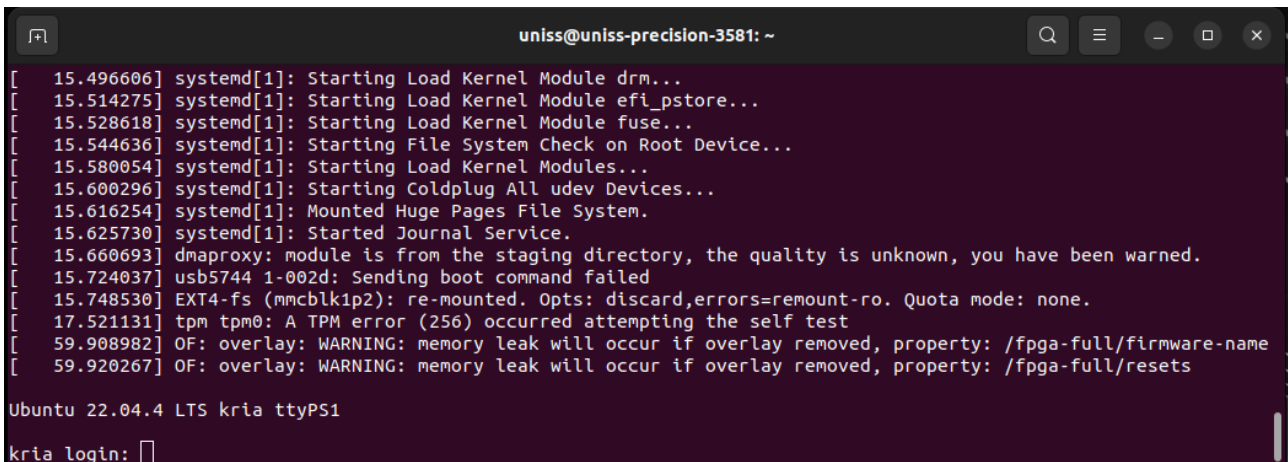


Figure 11: Kria KV260 board

3. (optional) [\[on your laptop\]](#) Control the board from via USB:

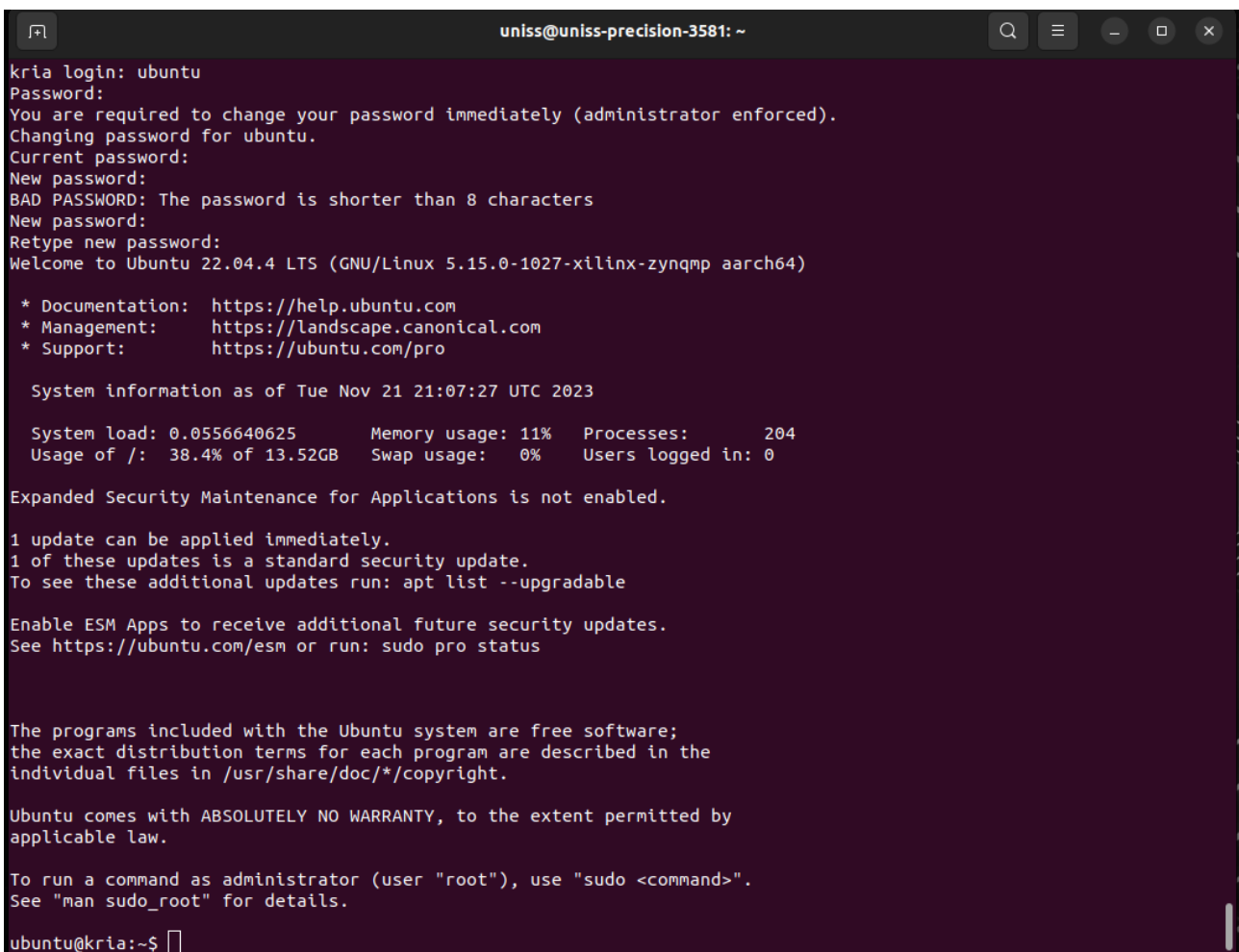
- Windows laptop: install Putty and open a serial connection (baudrate: 115200)
- Linux laptop: install picocom and run the following command (check if ttyUSB1 is related to the board): `sudo picocom -b 115200 /dev/ttyUSB1`

4. [\[on the board\]](#) Access the OS (user/pw: ubuntu) and change password (at least 8 characters)



```
uniss@uniss-precision-3581: ~  
[ 15.496606] systemd[1]: Starting Load Kernel Module drm...  
[ 15.514275] systemd[1]: Starting Load Kernel Module efi_pstore...  
[ 15.528618] systemd[1]: Starting Load Kernel Module fuse...  
[ 15.544636] systemd[1]: Starting File System Check on Root Device...  
[ 15.580054] systemd[1]: Starting Load Kernel Modules...  
[ 15.600296] systemd[1]: Starting Coldplug All udev Devices...  
[ 15.616254] systemd[1]: Mounted Huge Pages File System.  
[ 15.625730] systemd[1]: Started Journal Service.  
[ 15.660693] dmabproxy: module is from the staging directory, the quality is unknown, you have been warned.  
[ 15.724037] usb5744 1-002d: Sending boot command failed  
[ 15.748530] EXT4-fs (mmcblk1p2): re-mounted. Opts: discard,errors=remount-ro. Quota mode: none.  
[ 17.521131] tpm tpm0: A TPM error (256) occurred attempting the self test  
[ 59.908982] OF: overlay: WARNING: memory leak will occur if overlay removed, property: /fpga-full/firmware-name  
[ 59.920267] OF: overlay: WARNING: memory leak will occur if overlay removed, property: /fpga-full/resets  
  
Ubuntu 22.04.4 LTS kria ttyPS1  
kria login: 
```

Figure 12: login in Kria Ubuntu

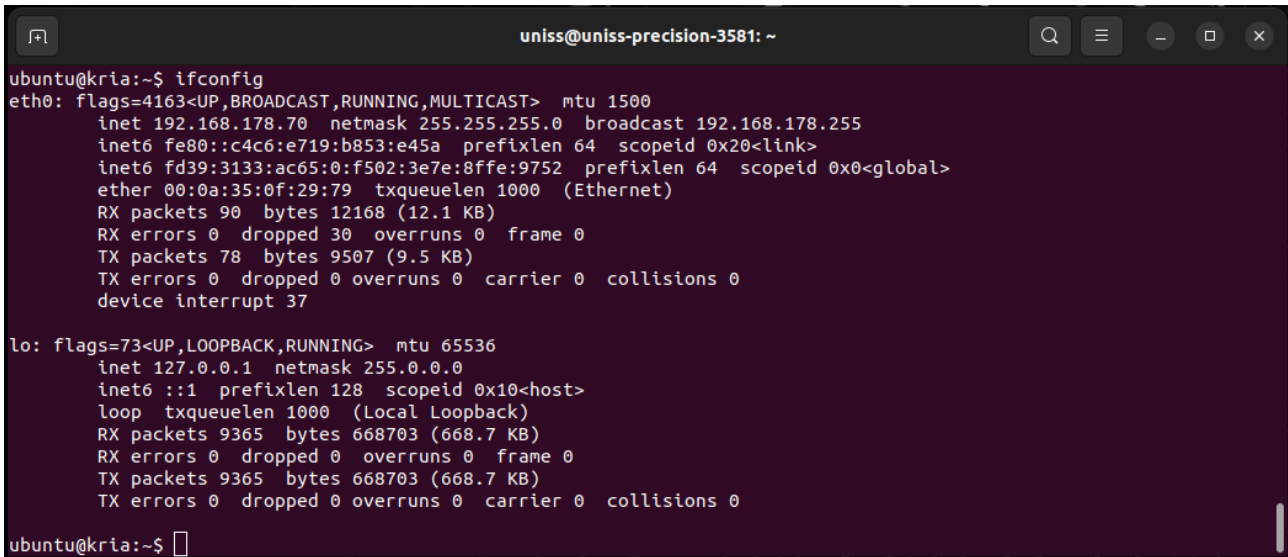


```
uniss@uniss-precision-3581: ~  
kria login: ubuntu  
Password:  
You are required to change your password immediately (administrator enforced).  
Changing password for ubuntu.  
Current password:  
New password:  
BAD PASSWORD: The password is shorter than 8 characters  
New password:  
Retype new password:  
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-1027-xilinx-zynqmp aarch64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/pro  
  
System information as of Tue Nov 21 21:07:27 UTC 2023  
  
System load: 0.0556640625      Memory usage: 11%      Processes:      204  
Usage of /: 38.4% of 13.52GB   Swap usage: 0%        Users logged in: 0  
  
Expanded Security Maintenance for Applications is not enabled.  
  
1 update can be applied immediately.  
1 of these updates is a standard security update.  
To see these additional updates run: apt list --upgradable  
  
Enable ESM Apps to receive additional future security updates.  
See https://ubuntu.com/esm or run: sudo pro status  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
To run a command as administrator (user "root"), use "sudo <command>".  
See "man sudo_root" for details.  
  
ubuntu@kria:~$ 
```

Figure 13: Kria Ubuntu is ready to use

5. (optional) Control the board from via Ethernet:

- [on the board] get the IP with the command (Figure 14): ifconfig
- [on your laptop] access through SSH (the IP must be the same you get with ifconfig)
`ssh ubuntu@192.168.178.70`



```
uniss@uniss-precision-3581: ~  
ubuntu@kria:~$ ifconfig  
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.178.70 netmask 255.255.255.0 broadcast 192.168.178.255  
    inet6 fe80::c4c6:e719:b853:e45a prefixlen 64 scopeid 0x20<link>  
    inet6 fd39:3133:ac65:0:f502:3e7e:8ffe:9752 prefixlen 64 scopeid 0x0<global>  
    ether 00:0a:35:0f:29:79 txqueuelen 1000 (Ethernet)  
    RX packets 90 bytes 12168 (12.1 KB)  
    RX errors 0 dropped 30 overruns 0 frame 0  
    TX packets 78 bytes 9507 (9.5 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
    device interrupt 37  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 9365 bytes 668703 (668.7 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 9365 bytes 668703 (668.7 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
ubuntu@kria:~$
```

Figure 14: get the IP with the command

6. [on the board] Clone the GitHub repo [AXI-based-driver-applications](https://github.com/mcheshfar/AXI-based-driver-applications) (or download the related ZIP file):

```
git clone https://github.com/mcheshfar/AXI-based-driver-applications.git
```

7. [on the board] Go to the folder `AXI-based-driver-applications/ubuntu-driver/` and run the following commands:

```
chmod +x set_env.sh
```

```
./set_env.sh
```

After the script execution, some services could require to be restarted: click on ESC

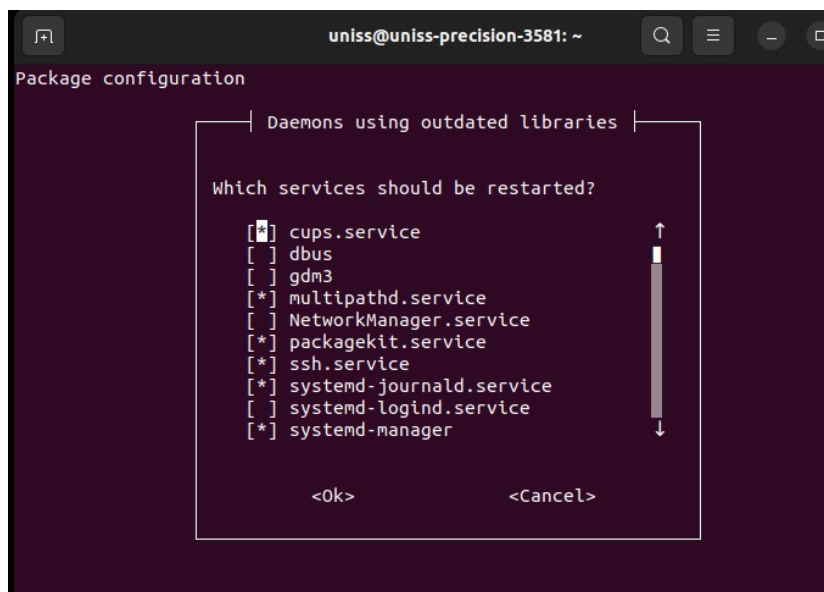


Figure 15: the script execution is finished

8. [\[on the board\]](#) Go to [AXI-based-driver-applications/kv260/applications/edgedetection/MDC](#) (the application folder) put your **design_1_wrapper.bit** beside other files. Then run the following commands:

```
gcc acc_test_edgedetect.c -o acc_test_edgedetect
sudo fpgauttil -R
sudo fpgauttil -b design_1_wrapper.bit
sudo fpgauttil -o pl_acc.dtbo
sudo ./acc_test_edgedetect roberts input.pgm
sudo ./acc_test_edgedetect sobel input.pgm
```

```
$ sudo ./acc_test_edgedetec roberts input.pgm
Input image (16x16):
5C 58 56 54 51 51 4F 50 4C 48 42 2F 2D 2A 42 52
77 5A 57 57 56 53 52 52 4F 4C 43 36 32 32 43 67
B8 77 58 56 56 53 52 52 4C 4C 40 38 32 4C 71 5E
D9 C5 83 56 52 53 51 4E 4E 4A 43 39 38 2F 4C 89
BD BC AD 96 50 4E 49 4B 47 4B 40 3A 38 33 2D 43
B1 A4 AD AE 6C 4F 4E 4C 4C 48 40 3B 40 62 AD EC
BF BE 7B BE 93 5B 52 4C 4D 4C 64 AB E3 F4 DF AB
80 A9 9F B8 8C 61 59 6D AC E8 F5 E5 B4 72 4C 4F
8A B4 93 B7 78 B1 E8 F6 EC BB 7C 4E 39 36 3B 4B
AF AC AF CE F6 EF CB 8C 5C 4F 47 3F 37 34 3A 48
A6 A6 A4 C3 9B 6A 56 52 51 4A 46 3F 38 36 39 48
A5 B3 BC AE 80 57 57 4E 4E 49 44 3D 32 33 36 46
9E AD B2 BA A6 70 4F 4E 4B 4A 46 3D 32 36 38 45
9E AA 82 AA B2 A3 5F 4B 49 46 41 3A 32 32 37 43
A1 AB BA C0 C6 C1 AD 5A 49 46 41 34 2D 33 39 44
BE C3 C8 CA CA CC CD AE 53 49 3F 33 2C 32 39 44

Accelerator configured: REG0=0x01000001, REG1=225
DMA transfers started.
DMA0 status = 0x00001002 -> DONE
DMA1 status = 0x00001002 -> DONE
DMA2 status = 0x00001002 -> DONE
out_pixels = 225
out_bytes = 900
out_ptr = 0xaaaac8b048b0
read completed

Roberts output (225 words):
FF 00 00 00 00 FF 00 00 00 00 00 00 00 00 00
FF 00 00 00 00 00 FF 00 00 00 00 00 00 00 00
FF 00 00 00 00 00 FF FF 00 00 00 00 00 00 00
FF 00 00 00 00 00 00 FF FF 00 00 00 00 00 00
00 FF 00 00 00 00 00 00 00 00 00 00 00 00 00
FF FF 00 00 00 00 00 00 00 00 00 00 00 00 00
FF FF FF 00 00 00 00 00 00 00 00 00 00 00 00
FF 00 FF FF 00 00 00 00 00 00 00 00 00 00 00
FF 00 00 FF 00 00 00 00 00 00 00 00 00 00 FF
FF 00 00 00 FF FF 00 00 00 00 00 00 FF FF FF
FF FF 00 FF FF FF 00 00 FF FF FF FF 00 FF FF
FF FF 00 00 FF 00 FF FF FF FF FF FF FF FF 00
FF 00 00 00 FF FF 00 FF FF FF FF FF 00 00 00
FF 00 00 00 FF FF FF FF 00 00 00 00 00 00 00
FF 00 00 00 FF FF 00 00 00 00 00 00 00 00 00
```

```
sudo ./acc_test_edgedetec sobel input.pgm
Input image (16x16):
5C 58 56 54 51 51 4F 50 4C 48 42 2F 2D 2A 42 52
77 5A 57 57 56 53 52 52 4F 4C 43 36 32 32 43 67
B8 77 58 56 56 53 52 52 4C 4C 40 38 32 4C 71 5E
D9 C5 83 56 52 53 51 4E 4E 4A 43 39 38 2F 4C 89
BD BC AD 96 50 4E 49 4B 47 4B 40 3A 38 33 2D 43
B1 A4 AD AE 6C 4F 4E 4C 4C 48 40 3B 40 62 AD EC
BF BE 7B BE 93 5B 52 4C 4D 4C 64 AB E3 F4 DF AB
80 A9 9F B8 8C 61 59 6D AC E8 F5 E5 B4 72 4C 4F
8A B4 93 B7 78 B1 E8 F6 EC BB 7C 4E 39 36 3B 4B
AF AC AF CE F6 EF CB 8C 5C 4F 47 3F 37 34 3A 48
A6 A6 A4 C3 9B 6A 56 52 51 4A 46 3F 38 36 39 48
A5 B3 BC AE 80 57 57 4E 4E 49 44 3D 32 33 36 46
9E AD B2 BA A6 70 4F 4E 4B 4A 46 3D 32 36 38 45
9E AA 82 AA B2 A3 5F 4B 49 46 41 3A 32 32 37 43
A1 AB BA C0 C6 C1 AD 5A 49 46 41 34 2D 33 39 44
BE C3 C8 CA CA CC CD AE 53 49 3F 33 2C 32 39 44

Accelerator configured: REG0=0x02000001, REG1=196
DMA transfers started.
DMA0 status = 0x00001002 -> DONE
DMA1 status = 0x00001002 -> DONE
DMA2 status = 0x00001002 -> DONE
out_pixels = 196
out_bytes = 784
out_ptr = 0xaaaad34b18b0
read completed

Sobel output (196 words):
FF 00 00 00 00 00 00 00 00 FF FF 00 FF FF
FF FF FF 00 00 00 00 00 00 00 00 00 FF FF
FF FF FF FF 00 00 00 00 00 00 00 00 FF FF
FF FF FF FF 00 00 00 00 00 00 00 FF FF FF
FF 00 FF FF FF 00 00 00 00 FF FF FF FF
FF 00 00 FF FF FF FF FF FF FF FF FF 00 FF
00 00 00 FF FF FF FF FF FF FF FF FF FF
FF FF FF FF FF FF FF FF FF FF FF FF FF
00 FF FF 00 FF FF FF FF FF FF FF 00 00 FF
00 FF FF FF FF FF FF FF FF 00 00 00 00 FF
FF 00 FF FF FF FF 00 00 00 00 00 00 00 00
FF FF 00 FF FF FF 00 00 00 00 00 00 00 00
00 00 FF FF FF FF FF 00 00 00 FF 00 00 00
FF FF FF FF FF FF FF FF 00 00 FF 00 00 00
```

Figure 16: run test code on Kria board (a) config select Robets filter (b) onfig select Sobel filter